

Lab 2 — Task 4: Docker Network

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Create a user-defined Docker bridge network and connect two containers to it, then verify container-to-container communication both by container name (via Docker's embedded DNS) and by internal IP address, without publishing any ports to the host.

Step 1 — Create a user-defined bridge network

A custom bridge network (lab2-net) is created, and two containers are attached to it without publishing any ports to the host: the Flask app (lab2-net-app) and a lightweight client container (lab2-net-client) running curlimages/curl in an idle sleep loop so it stays up for exec commands.

```
docker network create lab2-net
docker network inspect lab2-net --format "Name: {{.Name}} Driver: {{.Driver}}"
docker run -d --name lab2-net-app --network lab2-net lab2-python-app:1.0
docker run -d --name lab2-net-client --network lab2-net curlimages/curl:latest sleep 3600
docker ps --filter name=lab2-net
```

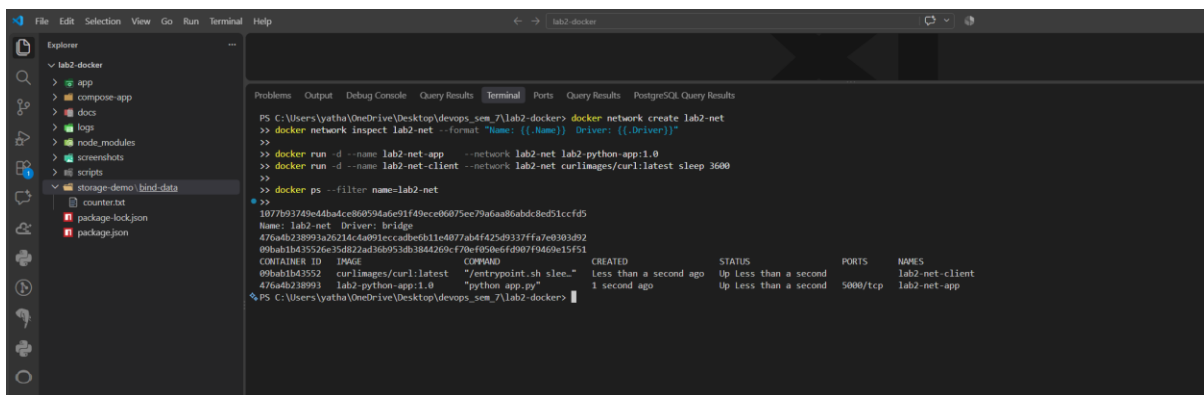


Figure 1 — lab2-net is a bridge-driver network; both containers report STATUS “Up” with no host ports published (5000/tcp shown as the container's internal port only).

Step 2 — Reach the app container by name

From inside lab2-net-client, the app is reached using its container name as the hostname (http://lab2-net-app:5000). This relies on the automatic DNS resolution that Docker provides on user-defined bridge networks.

```
docker exec lab2-net-client curl -s http://lab2-net-app:5000/
docker exec lab2-net-client curl -s http://lab2-net-app:5000/health
```

```
PS C:\Users\yatha\OneDrive\Desktop\devops_sen_7\lab2-docker> docker network create lab2-net
>> docker network inspect lab2-net --format '{{.Name}} {{.Driver}}'
>>
>> docker run -d --name lab2-net-app --network lab2-net lab2-python-app:1.0
>> docker run -d --name lab2-net-client --network lab2-net curlimages/curl:latest sleep 3600
>>
>> docker ps --filter name=lab2-net
*
10/7893749e44ba6ce8609946e91f49ce06075ee79aba86badc8ed51ccfd5
Name: lab2-net Driver: bridge
476ad238993lab218c4a991eccead8b11c4077d4d42e40137f1fae0380802
09b611b43552e1c5d0225d80953b3844269cf70cf09d6f6807f9469e15f51
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
09b611b43552 curlimages/curl:latest "/entrypoint.sh slee..." Less than a second ago Up Less than a second 5000/tcp lab2-net-client
476ad238993 lab2-python-app:1.0 "python app.py" 1 second ago Up Less than a second 5000/tcp lab2-net-app
PS C:\Users\yatha\OneDrive\Desktop\devops_sen_7\lab2-docker> docker exec lab2-net-client curl -s http://lab2-net-app:5000/health
{"data_dir":"/data","hostname":"476ad238993","message":"Hello from the Lab 2 containerized Python app!","time_utc":"2026-09-08T07:04:17.931224+00:00","visits_recorded_in_this_container":1}
{"status":"ok"}
% PS C:\Users\yatha\OneDrive\Desktop\devops_sen_7\lab2-docker>
```

Figure 2 — Both requests succeed by container name alone; the `/` response reports the app's hostname and visit count, and `/health` returns `{"status":"ok"}`.

Step 3 — Reach the app container by IP address

For comparison, the app container's IP address on `lab2-net` is retrieved with `docker inspect`, and the same request is repeated from the client using the raw IP instead of the container name.

```
docker inspect -f '{{(index .NetworkSettings.Networks "lab2-net").IPAddress}}' lab2-net-app
docker exec lab2-net-client curl -s http://172.20.0.2:5000/
```

```
PS C:\Users\yatha\OneDrive\Desktop\devops_sen_7\lab2-docker> docker inspect -f '{{(index .NetworkSettings.Networks "lab2-net").IPAddress}}' lab2-net-app
172.20.0.2
>> docker exec lab2-net-client curl -s http://172.20.0.2:5000/
{"data_dir":"/data","hostname":"476ad238993","message":"Hello from the Lab 2 containerized Python app!","time_utc":"2026-09-08T07:04:49.531658+00:00","visits_recorded_in_this_container":2}
% PS C:\Users\yatha\OneDrive\Desktop\devops_sen_7\lab2-docker>
```

Figure 3 — `lab2-net-app` resolves to `172.20.0.2` on the custom network; the client reaches it directly by IP with the same JSON response, now showing `visits_recorded_in_this_container: 2`.

Result

A user-defined bridge network let two containers communicate with each other directly — by container name via Docker's built-in embedded DNS, and equally by internal IP address — without publishing any ports to the host. This confirms that container-to-container traffic on a custom network is isolated from, and independent of, host port mappings. Task 4 is complete.